

DamageScope: Vision–Language Retrieval at Scale for Disaster Damage Assessment from Satellite Imagery

Ravi K. Rajendran*, Biplob Debnath*, Murugan Sankaradas† and Srimat T. Chakradhar
NEC Laboratories America, Princeton, NJ, USA

Email: {rarajendran,biplob,murugs,chak}@nec-labs.com

Abstract

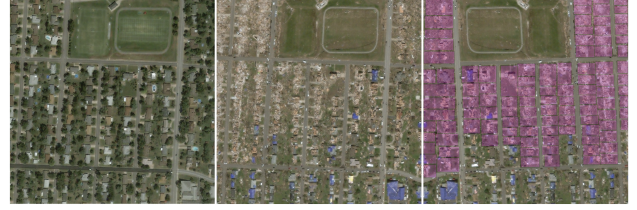
Timely and accurate assessment of property damage is critical following natural disasters. Traditional on-site inspections are labor-intensive, costly, and often pose safety risks. Advances in satellite imagery and vision-language models (VLMs) enable scalable remote damage assessment; however, integrating VLMs into large-scale Earth observation pipelines presents challenges in computational efficiency, data organization, and information retrieval. To address these challenges, we present DamageScope, a retrieval-augmented framework that combines satellite imagery with Vision-Language Models (VLMs) and Large Language Models (LLMs) to automate property damage analysis. Built on a Retrieval-Augmented Generation (RAG) framework, DamageScope extracts structured visual representations from satellite imagery to support interactive natural language queries for damage assessment. To address scalability, we introduce a novel multi-vector embedding-based clustering algorithm that outperforms traditional single-vector embedding approaches while reducing indexing time by up to 14 $\times$. Furthermore, a dual-store data architecture minimizes LLM API calls, reducing both operational cost and response latency by up to approximately 3 $\times$. By effectively balancing scalability and operational efficiency, DamageScope provides a robust and practical solution for real-world damage assessment tasks.

1. Introduction

Accurate and timely assessments of property damage play a critical role in disaster recovery, particularly in the aftermath of natural disasters such as hurricanes, earthquakes, or floods. These evaluations are essential for determining the extent of damage, expediting insurance claims, and facilitating the restoration process for affected individuals and communities. However, traditional methods of damage as-

* Equal contribution.

† Work performed while at NEC Laboratories America.



Query: How many properties are severely damaged?
Response: 103 properties (out of 138 properties) are severely damaged.

Figure 1. DamageScope’s response to the query, “List all the properties that are severely damaged.” The figure shows the pre- (left), post-disaster image (middle), and annotated response highlighting the damaged properties (right).

essment primarily rely on on-site inspections conducted by field adjusters. While this approach has been the standard practice for years, it comes with several challenges and limitations. On-site inspections are often costly due to the resources required, leading to delays in processing claims and increased operational expenses. Additionally, they can be time-consuming, as adjusters must physically visit each affected property to document and evaluate the damage. This process can be further delayed when access to certain areas is restricted due to infrastructure damage or hazardous conditions caused by the disaster. Given these limitations, there is a growing need for more efficient and safer alternatives to traditional damage evaluation methods.

Recent advancements in satellite imaging and artificial intelligence (AI) are transforming this damage assessment process. Satellite imagery enables insurers to conduct rapid damage assessments over extensive areas without the need for physical on-site visits. High-resolution satellite images allow insurers to quickly gauge the extent of damage across different types of properties - be it structural, roof, or environmental damage while minimizing human intervention [2, 30]. The integration of AI with satellite data further enhances this process by automating the analysis of imagery to detect and quantify damage. Machine learning models, particularly deep learning techniques such as convolutional neural networks (CNNs) and transformers, have demon-

strated high accuracy in damage classification tasks [5, 12]. More recently, vision-language models (VLMs) that fuse satellite images with metadata have been used to improve assessment precision. These advancements enable insurers to process claims more quickly, make well-informed decisions with minimal delays, and ultimately improve the overall customer experience.

In this paper, we introduce *DamageScope*, a system designed to enhance property damage analysis by integrating satellite imagery, AI models, and large language models (LLMs). *DamageScope* is built on the retrieval-augmented generation (RAG) framework [18], enabling efficient retrieval and reasoning over damage data. Damage analysis begins with a satellite image as the primary input. The system uses property metadata, such as geographical boundaries and property records, to isolate individual properties within the image. Once each property image is extracted, it is processed by a vision-language model (VLM), such as GPT-4o [21], which is capable of analyzing both visual data and corresponding contextual information. The VLM analyzes an image to extract various damage-related metadata, including structural integrity, roof damage, water accumulation, habitability status, and overall damage score. This extracted information is then saved in data stores for later querying.

DamageScope allows insurers to interact with the system using natural language queries, facilitating seamless access to critical insights. Given a satellite image, now users can ask detailed questions about damage severity, affected regions, or specific property conditions. The system retrieves relevant information from the data store and provides it as context to an LLM [21] along with the query, which then generates a comprehensive, context-aware response. For example, Figure 1 illustrates the *DamageScope*’s output for a query identifying “severely damaged properties”.

Challenges. Deploying the *DamageScope* system presents several challenges that must be addressed to ensure efficiency, scalability, and accuracy in property damage assessment.

Challenge #1: Computational Efficiency in Damage Metric Generation. Extracting and analyzing damage metrics from property images is a computationally intensive task, particularly in large-scale disaster scenarios involving thousands of properties. Processing each image individually using VLMs is prohibitively slow and expensive. To address this, *DamageScope* employs a clustering-based strategy that groups properties with similar characteristics, thereby reducing redundant computation.

While traditional clustering methods typically rely on single-vector embeddings, we utilize *multi-vector embeddings* [7, 15, 25], which have shown superior performance in information retrieval tasks [9, 23, 25]. To quantify

similarity between two multi-vector representations, the MaxSim function [7, 9, 15] is widely used, as it identifies the maximum token-level similarity across the two sets of vectors. However, MaxSim is inherently asymmetric, i.e., $\text{MaxSim}(A, B) \neq \text{MaxSim}(B, A)$, which limits its applicability in clustering algorithms that require symmetric distance measures.

To overcome this limitation, we propose a *symmetric variant* of the MaxSim function. By averaging the directional MaxSim scores between two sets of vectors, the resulting $\text{MaxSim}_{\text{sym}}$ yields a reciprocal similarity measure, enabling consistent distance computations essential for clustering.

Challenge #2: Efficient Organization and Retrieval of Damage Information. Once damage metrics are generated, it is essential to structure the data in a way that facilitates efficient retrieval. Storing all damage information within a single store can lead to slow and inaccurate retrieval processes. To address this challenge, *DamageScope* employs a multi-store data organization strategy, distributing the damage data across multiple data stores, thus optimizing retrieval time and accuracy.

Challenge #3: Optimized Query Processing with Large Language Models. *DamageScope* leverages LLMs to generate concise, human-like responses to user queries. However, it faces two main challenges: (1) LLMs are constrained by context size limitations, restricting the amount of input they can process at any given time, and (2) the cost of using the LLM API can be high [1]. To overcome these challenges, *DamageScope* employs a strategic data filtering mechanism leveraging LLMs. Rather than feeding the entire data to be processed, the *DamageScope* uses LLM-driven planning to selectively retrieve the most relevant information from the data store to form the context for response generation, optimizing both performance and cost-efficiency.

By addressing these challenges - reducing computational costs through clustering, optimizing data retrieval through multiple stores, and improving query efficiency through context-aware LLM integration, *DamageScope* achieves a balance between accuracy, scalability, and efficiency. In summary, we make the following contributions:

- We introduce *DamageScope* system that integrates satellite imagery, Vision Language Models (VLMs), and Large Language Models (LLMs) to perform property damage assessment. It supports natural language querying, allowing users to retrieve damage insights.
- We present a clustering algorithm based on multi-vector embeddings to improve the computational efficiency of property-level damage information generation. Evaluation on the xBD [31] benchmark shows that our multi-

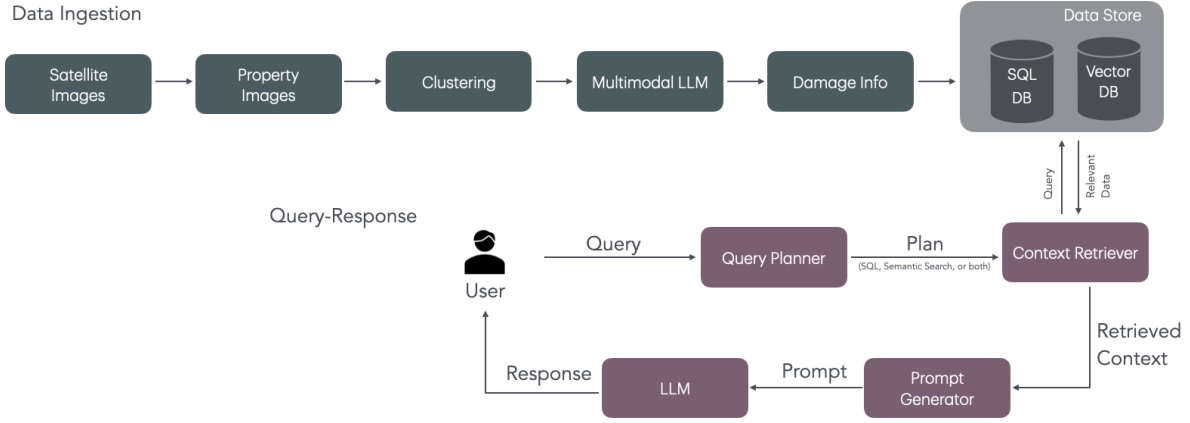


Figure 2. DamageScope workflow. The ingestion pipeline (top) converts satellite imagery into structured and semantic representations stored in a dual data store. The query-response pipeline (bottom) retrieves relevant evidence and composes an LLM-grounded response.

vector clustering approach outperforms its single-vector counterpart, achieving gains of up to 0.20 in Normalized Mutual Information (NMI) [29], 0.26 in Fowlkes-Mallows Score (FMS) [10], and 0.26 in Adjusted Rand Index (ARI) [14]. Additionally, it reduces overall indexing time up to a factor of $14\times$.

- Our evaluation shows that the question-answering capability of DamageScope is approximately $3\times$ more cost-efficient and $2.9\times$ faster compared to baseline approaches. This performance gain is achieved through a dual-store data organization that distributes the data across multiple data stores, optimizing retrieval latency and resource efficiency.

2. DamageScope System

DamageScope builds on the Retrieval-Augmented Generation (RAG) paradigm, integrating multimodal perception and language-based reasoning to support scalable, interpretable damage assessment. The system is designed around the real-world requirements of *speed*, *scalability*, and *traceability*. It operates in two stages: (1) data ingestion, which converts raw satellite imagery into structured representations, and (2) question answering, which retrieves relevant evidence and synthesizes natural-language responses. An overview of the DamageScope system workflow is shown in Figure 2.

2.1. Data Ingestion: Extracting Structured Damage Information

Traditional satellite pipelines use pixel-level models [5, 31], producing narrow outputs that are hard to reuse for reasoning or retrieval. DamageScope instead generates a semantically rich representation for each property, combining categorical attributes (e.g., roof integrity, structural damage) with natural-language rationales grounded in the imagery.

In DamageScope, analysis begins with a satellite image and property metadata, which define geographic boundaries and allow extraction of individual property images. Each property crop is then processed by a vision-language model (VLM) prompted to return a JSON summary with categorical predictions and natural-language reasoning.

A sample output is shown in Listing 1. Such structured outputs enable both factual querying (e.g., `damage_level`) and semantic retrieval over reasoning. However, naively invoking a VLM on every building introduces significant computational overhead: a single GPT-4o [21] inference takes ≈ 8.4 seconds. For a mid-sized city with 10,000 building footprints, this translates to ~ 23 hours of sequential processing, which is far too slow for rapid post-disaster assessment. Although parallelization can reduce wall-clock latency, it does not address the underlying scalability and cost constraints inherent to a “VLM-per-property” pipeline. The next section describes how DamageScope addresses this bottleneck through clustering.

Listing 1. Example structured damage summary.

```
{
  "property_id": "A12-487",
  "roof_damage": {
    "prediction": true,
    "reason": "Significant portions of the roof shingles
               ↪ are missing."
  },
  "damage_level": {
    "prediction": 3,
    "reason": "Major roof loss with likely internal damage.
               ↪ "
  }
}
```

2.1.1. Clustering for Scalable Ingestion

Disaster damage exhibits spatial correlation: nearby properties often share similar destruction patterns. To avoid re-



Figure 3. Damage information generation workflow using clustering. First, images are classified into four groups. Next, the “No Damage” group contains a single cluster, while the other groups are further clustered using multi-vector embeddings. Damage information is then generated for each cluster centroid using a LLM. Finally, damage information is stored in the data store.

dundant VLM inference, DamageScope performs clustering and analyzes only representative items from each cluster.

To form clusters, DamageScope uses a two-step process as shown in Figure 3. Each property image is classified into one of four categories: 0 (No Damage), 1 (Minor), 2 (Major), or 3 (Destroyed), using labeled data such as the xBD dataset [11]. Properties labeled “No Damage” require no further analysis. For all other classes, DamageScope performs clustering within each category to capture finer-grained, structurally similar damage patterns.

This two-step pipeline, consisting of (1) coarse classification followed by (2) intra-class clustering, improves both efficiency and representational quality. The initial classifier reduces heterogeneity by partitioning images into coarse severity groups, creating more homogeneous subsets for clustering. Applying clustering only within non-zero classes then reveals coherent, fine-grained damage patterns that would be obscured in a single global clustering step. This design reduces noise, improves cluster stability, and avoids unnecessary computation for undamaged properties, enabling more interpretable and scalable analysis.

2.1.2. Intra-Class Clustering

Multi-Vector Embedding. Single-vector embeddings compress spatial information into a single point, making them insufficient for capturing fine-grained cues such as shingle loss, debris distribution, or partial collapse. DamageScope therefore adopts multi-vector embeddings, which preserve token-level information across the image grid and have demonstrated strong performance in late-interaction retrieval models [7, 15, 23, 25].

A challenge arises because standard late-interaction similarity scoring functions, such as MaxSim [15], are *asymmetric*:

$$\text{MaxSim}(A, B) \neq \text{MaxSim}(B, A)$$

as the score averages over tokens from only one embedding set. While acceptable for ranking applications, this asym-

metry makes MaxSim unsuitable as a *distance function* for clustering. In general, clustering algorithms, whether hierarchical [27], centroid based (e.g., k-means variants [20]), or density based [6], require a *symmetric* pairwise distance measure; otherwise, the resulting distance matrix becomes ill formed and produces inconsistent cluster assignments that depend on arbitrary ordering rather than true similarity.

To enable clustering with multi-vector representations, we introduce a symmetric extension of MaxSim. Given multi-vector embeddings $A = \{a_i\}_{i=1}^m$ and $B = \{b_j\}_{j=1}^n$, and a similarity function $\text{sim}(\cdot, \cdot)$ (e.g., cosine similarity), we define:

$$\text{MaxSim}_{\text{sym}}(A, B) = \frac{1}{2} \left(\frac{1}{m} \sum_i \max_j \text{sim}(a_i, b_j) + \frac{1}{n} \sum_j \max_i \text{sim}(b_j, a_i) \right) \quad (1)$$

This formulation guarantees:

$$\text{MaxSim}_{\text{sym}}(A, B) = \text{MaxSim}_{\text{sym}}(B, A)$$

allowing us to define a valid distance metric

$$d(A, B) = 1 - \text{MaxSim}_{\text{sym}}(A, B).$$

Using this symmetric metric produces coherent, stable clusters that reflect mutual visual similarity between damaged properties. In practice, this improves cluster purity and provides a reliable basis for reducing redundant ingestion computations.

Clustering Algorithm. DamageScope clusters embedded properties and sends only one representative per cluster to the VLM. It uses hierarchical agglomerative clustering (HAC) [27] because it aligns with the structure of multi-vector embeddings. Damage patterns often form *non-convex*, *heterogeneous*, and *uneven-sized* groups, conditions under which k-means [20] fails due to its spherical-

cluster assumption. Density-based methods such as DBSCAN [6] also struggle with scale-sensitive thresholds in high-dimensional token spaces. HAC naturally supports our symmetric pairwise distance $d(A, B)$ without requiring centroid computation and better preserves semantic neighborhoods. For a class with N_c samples, we set the number of clusters to

$$K_c = \lfloor \sqrt{N_c} \rfloor,$$

following heuristics from ColBERTv2 [24].

2.1.3. Dual-Store Data Organization

Ingestion produces two distinct information types: (i) *structured attributes* (e.g., damage indicators, severity levels) and (ii) *textual reasoning* describing visual evidence. Treating these uniformly is inefficient: relational databases excel at structured filtering but perform poorly on semantic search, while vector stores support embedding-based retrieval but are inefficient for grouping, aggregation, and large-scale filtering.

To address this, DamageScope uses a dual-store architecture that assigns each data type to the system best suited for it. Structured metadata is stored in SQL database (DB), enabling fast indexing and precise queries such as “*Filter all properties with damage_level ≥ 2* ” or “*List properties with structural damage*”. Textual reasoning is embedded and stored in a vector DB, supporting open-ended retrieval such as “*Find properties with signs of roof shingle loss*” or “*Retrieve all houses described as uninhabitable*”.

Listing 2. Structured metadata stored in SQL DB.

```
{
  "property_id": "A12-487",
  "roof_damage": true,
  "damage_level": 3
}
```

The SQL database stores compact, schema-aligned records, as shown in Listing 2. The vector database retains semantically rich reasoning information, as illustrated in Listing 3. By routing each query to the appropriate store, DamageScope avoids unnecessary text processing and prevents large reasoning fields from inflating the LLM context window.

Listing 3. Reasoning fields stored in the vector DB.

```
{
  "property_id": "A12-487",
  "roof_damage_reason":
    "Significant portions of the roof shingles are missing.
     ⇨ ",
  "damage-level_reason":
    "Major roof loss with likely internal damage"
}
```

2.2. Question Answering: Retrieval Aware LLM Reasoning

After ingestion, users query DamageScope in natural language (see the bottom part of Figure 2). These queries leverage the two data types defined in the dual-store architecture described in Section 2.1.3: structured attributes (e.g., damage_level) and textual reasoning fields (e.g., roof_damage_reason). Sending all fields to an LLM is impractical due to context length limitations and cost constraints.

To manage this, DamageScope employs a retrieval aware query planner that determines which store, SQL or vector, is required for a given question. Metadata oriented queries (e.g., “*List properties with damage_level ≥ 2* ”) are translated into SQL filters, while descriptive queries (e.g., “*Find houses with missing shingles*”) trigger vector similarity search over reasoning embeddings. Questions that require both evidence types retrieve from both stores. DamageScope then assembles a compact prompt containing the user query and only the relevant structured fields and reasoning snippets (e.g., roof_damage=true or “Significant portions of the roof shingles are missing”). This ensures grounded responses while keeping token usage and latency low.

2.3. Design Insights

DamageScope’s efficiency arises from the interaction of its components rather than any single mechanism. Multi-vector clustering reduces ingestion cost by exploiting spatial correlation in damage patterns, allowing the system to process only a small set of representative properties. The dual store architecture streamlines retrieval by separating fast relational queries from semantic similarity search so that each query retrieves only the evidence it requires. The LLM based query planner coordinates these capabilities by interpreting user intent, issuing appropriate SQL and vector queries, and keeping LLM context small and responses traceable. Together, these choices reflect a broader principle: combining retrieval aware reasoning with domain specific data structuring enables large language models to operate efficiently and reliably in high stakes, non textual settings such as post disaster damage assessment.

3. Experiments

In this section, we evaluate the quality of the clusters, the impact of clustering, data organization, and the LLM API usage cost optimization of the DamageScope system.

3.1. Implementation Details

Data Ingestion: The ingestion phase begins with data pre-processing, where satellite images are processed to extract property information rather than just building details. This

Table 1. Clustering performance comparison between single and multi-vector embeddings using **RemoteCLIP** and **SigLIP** models. Green cells indicate best per method and metric; **bold** indicates best overall per damage type.

Image ID	Damage	Method	RemoteCLIP			SigLIP		
			NMI	FMS	ARI	NMI	FMS	ARI
31	Minor	Single-vector + kMeans	0.428	0.283	0.177	0.372	0.216	0.104
		Multi-vector + UPGMA	0.561	0.521	0.416	0.337	0.338	0.096
31	Major	Single-vector + kMeans	0.507	0.522	0.288	0.787	0.658	0.521
		Multi-vector + UPGMA	0.704	0.681	0.544	0.724	0.733	0.516
31	Destroyed	Single-vector + kMeans	0.543	0.366	0.225	0.544	0.399	0.239
		Multivector + UPGMA	0.561	0.567	0.365	0.529	0.526	0.393
16	Minor	Single-vector + kMeans	0.526	0.435	0.301	0.549	0.384	0.265
		Multi-vector + UPGMA	0.694	0.669	0.565	0.376	0.333	0.114
16	Major	Single-vector + kMeans	0.573	0.473	0.331	0.493	0.413	0.265
		Multi-vector + UPGMA	0.519	0.514	0.271	0.610	0.575	0.380
16	Destroyed	Single-vector + kMeans	0.422	0.272	0.132	0.432	0.257	0.126
		Multi-vector + UPGMA	0.446	0.428	0.216	0.421	0.477	0.218

preprocessing ensures that assessments account for the entire property, capturing a more comprehensive view of disaster impacts. Each property is identified and segmented for downstream analysis. Damage information is extracted using GPT-4o [21], which analyze and annotate the images for damage metrics (e.g., roof damage, water damage, etc.) and textual reasoning. To store the extracted data, we use dual-storage mechanisms to optimize query response time and accuracy. For damage metrics, we used SQLite [13]. For textual reasoning, we used ChromaDB [4] for vector storage. This approach ensures that SQL queries and semantic searches are executed efficiently.

Question Answering (QA): QA phase relies on the dual storage mechanism to optimize query response time and accuracy. Responses are generated using GPT-4o, which is used to make query plans, SQL queries, and other retrieval prompts. GPT-4o’s ability to reason about complex queries ensures that relevant information is fetched effectively from the data stores, thus minimizing unnecessary computations and enhances overall system performance.

To evaluate the effectiveness of DamageScope, we compare against a *baseline* system that does not use clustering for ingestion, uses a single data store, and does not perform query-aware content selection during question answering.

3.2. Dataset

For our experiments, we utilize a publicly available disaster impact dataset, including satellite imagery and property damage assessments. Specifically, we used xBD dataset [11], which provides pre- and post-disaster satellite images of properties affected by natural disasters such as hurricanes, tornadoes, etc., as well as proprietary customer data

to enhance the robustness of our evaluation. xBD dataset focuses on structural damage but includes only building polygons, lacking information about the entire property. To address this limitation, we performed additional preprocessing and annotation to delineate complete property boundaries. These boundaries encompass not only the building but also surrounding features such as lawns, backyards, front yards, and driveways, providing a more holistic view of disaster impact.

3.3. Results

3.3.1. Clustering Performance Evaluation

Table 1 presents a comparative analysis of clustering performance across different damage types and embedding strategies, using images from the Joplin tornado subset of the xBD dataset [31]. We evaluate the effectiveness of two vision-language models: RemoteCLIP [19] (chendelong/RemoteCLIP ViT-B-32) and SigLIP [33] (google/siglip-so400m-patch14-384). Images are processed using the vision encoder, and we extract the output from the final transformer layer to obtain token-level image features.

We use three standard clustering metrics: Normalized Mutual Information (NMI) [29], Fowlkes-Mallows Score (FMS) [10], and Adjusted Rand Index (ARI) [14]. These metrics assess how well the predicted clusters align with the ground truth labels generated from manual inspections.

We compare two primary clustering methods: (1) *Single-vector + kMeans*, which applies the kMeans [20] algorithm to average-pooled image embeddings*, and (2) *Multi-vector*

*We exclude results using the [CLS] token embedding, as they consistently underperform in our evaluations.

+ *UPGMA* [27], which applies hierarchical agglomerative clustering (HAC) with ‘average’ linkage to token-level embeddings using a symmetric MaxSim-based similarity measure. We use the scikit-learn clustering library [22] to implement and evaluate clustering algorithms. Results are grouped by image ID (16 or 31) and damage category (Minor, Major, Destroyed).

Key Observations:

- **Multi-vector methods consistently outperform single-vector baselines for RemoteCLIP across most cases.** For example, in the *Minor damage* category for Image ID 31, *Multi-vector* + *UPGMA* with RemoteCLIP achieves NMI = 0.561, FMS = 0.521, and ARI = 0.416, outperforming the single-vector baseline (NMI = 0.428, FMS = 0.283, ARI = 0.177) by substantial margins. Similar trends are observed in the Major and Destroyed categories as well.
- **RemoteCLIP benefits more from multi-vector representations than SigLIP.** For Image ID 16 in the Minor category, RemoteCLIP’s multi-vector approach significantly outperforms the single-vector baseline (NMI = 0.694 vs. 0.526; FMS = 0.669 vs. 0.435; ARI = 0.565 vs. 0.301), while SigLIP shows only modest or inconsistent gains. This suggests RemoteCLIP better encodes spatial and semantic details that can be exploited by multi-vector models.
- **SigLIP achieves the highest scores in a few settings, but the advantage is inconsistent.** For instance, in the Major category of Image ID 31, SigLIP’s single-vector method attains the best overall performance (NMI = 0.787, ARI = 0.521), slightly outperforming the multi-vector method. However, RemoteCLIP generally performs better across most other cases when paired with multi-vector embeddings.
- **Hierarchical agglomerative clustering algorithm UPGMA with multi-vector embeddings yields better alignment with true damage clusters.** This is particularly evident in the Destroyed category for both image IDs. For example, RemoteCLIP achieves ARI = 0.393 using multi-vector UPGMA clustering for Image ID 31, compared to 0.239 with the single-vector approach.

Overall, these results demonstrate that **multi-vector embeddings combined with hierarchical agglomerative clustering algorithm (i.e., UPGMA) offer superior clustering performance**, particularly when paired with RemoteCLIP, which has been finetuned for satellite imagery [19]. While single-vector methods remain competitive in certain scenarios, especially with SigLIP (despite its lack of finetuning for satellite images), they often struggle to capture the fine-grained spatial patterns essential for

Table 2. Data Ingestion Time

Image ID	Method	Time taken (seconds)
31	Baseline (w/o clustering)	2436.1
	DamageScope	172.9 ($14.08 \times$ faster)
16	Baseline (w/o clustering)	1335.6
	DamageScope	168.3 ($7.94 \times$ faster)

accurate damage assessment. These findings underscore the value of multi-vector embeddings in strengthening post-disaster analysis pipelines.

3.3.2. Effectiveness of Clustering for Ingestion

As shown in Table 2, DamageScope substantially reduces data ingestion latency by clustering spatially related damage instances. For Image 31, ingestion time decreases from 2,436.1 s without clustering to 172.9 s with clustering, corresponding to a $14.08\times$ speedup. Similarly, for Image 16, ingestion time decreases from 1,335.6 s to 168.3 s, yielding a $7.94\times$ speedup.

Interestingly, the baseline runtime does not scale directly with the number of property instances. Image 31 contains 139 instances, fewer than the 149 instances in Image 16, yet requires substantially more processing time (2,436.1 s vs. 1,335.6 s). The two images differ considerably in their damage composition: Image 31 contains 86 minor, 15 major, and 38 destroyed instances, whereas Image 16 contains 30 minor, 25 major, and 94 destroyed instances. This suggests that ingestion latency depends not only on the number of properties but also on characteristics of the underlying scene, such as the spatial distribution and visual complexity of the damage instances, as well as variability in model processing and API latency.

Clustering substantially reduces this instance-level workload. The 139 damage instances in Image 31 are consolidated into 20 clusters, while the 149 instances in Image 16 are reduced to 22 clusters. The key benefit of clustering is that it reduces the number of regions requiring individual processing, thereby lowering the overall ingestion workload. Consequently, DamageScope improves data ingestion efficiency and supports more scalable property-level damage analysis for disaster recovery and insurance applications.

3.3.3. QA Performance Gains in Cost and Latency

Our results demonstrate notable performance improvements of the DamageScope system over the baseline, particularly in terms of cost efficiency and query processing speed. As detailed in Table 3, DamageScope reduces the average LLM token usage from 7,050 to 2,420 tokens per query, achieving a $2.9\times$ reduction enabled by query-aware retrieval and compact context construction. This reduction not only makes the system more cost-effective but also demonstrates that

Table 3. Question-Answering: LLM API usage cost

	Cost (tokens)
Baseline	7050
DamageScope	2420 ($2.9 \times \text{cheaper}$)

Table 4. Question-Answering: Effect of Data Organization

	Latency (seconds)
Baseline	2.1
DamageScope	0.69 ($3 \times \text{faster}$)

DamageScope manages the contextual information required for answering queries more efficiently.

In terms of query latency, DamageScope outperforms the baseline at every stage of the query processing pipeline. As shown in Table 4, the end-to-end QA latency decreases from 2.10 seconds to 0.69 seconds, representing an approximately $3\times$ improvement over the baseline. DamageScope achieves these latency gains by generating optimized query plans and executing them more efficiently through a dual-store data architecture.

4. Related Work

Recent advances in satellite imagery and AI have significantly enhanced post-disaster damage assessment. Early efforts used high-resolution satellite data and object-based image analysis to detect structural damage from events like earthquakes and floods [28, 32].

Machine learning has further advanced this field. CNN-based models trained on datasets like xBD [31] have been used to classify damage, with improvements from multi-task learning [5] and ensemble methods [26]. More recent approaches extract both spectral and spatial features via dual-path networks [30]. Multimodal techniques, such as attention-based networks using pre- and post-disaster imagery with metadata [12], or transformers fusing radar and optical data [3] have improved accuracy. Vision-language models like GeoChat enable natural language interaction with geospatial data [17], while fusion frameworks combining hyperspectral and LiDAR data further enhance analysis [16]. AI-driven decision support systems now aid real-time disaster response. Tools like FEMA’s Damage Assessment Toolkit [8] and automated resource planning frameworks [34] show the growing integration of generative AI in operational settings.

DamageScope builds on this foundation by combining multimodal AI, LLMs, and retrieval-augmented generation (RAG) to enable fast, cost-efficient, and interactive satellite-based damage assessment for insurers.

5. Conclusion

In this paper, we present DamageScope, a system designed to transform property damage assessment in the insurance industry by leveraging satellite imagery and AI-driven analysis. Traditional damage assessment methods rely on physical inspections, which are often costly, time-consuming, and hazardous. DamageScope addresses these challenges by automating the evaluation process using high-resolution satellite imagery, vision-language models, and the generative capabilities of large language models. By enabling rapid, large-scale assessments, DamageScope helps insurers streamline claims processing, enhance accuracy, and provide more efficient services to policyholders in the aftermath of natural disasters.

References

- [1] Md Adnan Arefeen, Biplob Debnath, and Srimat Chakradhar. Leancontext: Cost-efficient domain-specific question answering using llms. *Natural Language Processing Journal*, 7:100065, 2024. 2
- [2] Daniel Brunner, Guido Lemoine, and Lorenzo Bruzzone. Earthquake damage assessment based on object-based analysis of vhr satellite images. *IEEE Transactions on Geoscience and Remote Sensing*, 2010. 1
- [3] Ilias Chamatidis, Denis Istrati, and Nikos D. Lagaros. Vision transformer for flood detection using satellite images from sentinel-1 and sentinel-2. *Water*, 16(12), 2024. 8
- [4] ChromaDB. Chromadb: A high-performance vector database, 2023. 6
- [5] Jigar Doshi, Saikat Basu, and Guan Pang. From satellite imagery to disaster insights, 2018. 2, 3, 8
- [6] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, pages 226–231, 1996. 4, 5
- [7] Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. Colpali: Efficient document retrieval with vision language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025. 2, 4
- [8] Federal Emergency Management Agency. Preliminary damage assessments, 2024. Accessed on February 11, 2025. 8
- [9] Tomer Formal, Keshav Santhanam, Biswajit Paria, Ranjie You, Michael Wornow, Bo Pang, Balasubramaniam Narayanaswamy, and Benjamin Van Durme. Xtr: Cross-token representations for efficient and effective dense retrieval. In *Proceedings of the Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 2
- [10] Edward B Fowlkes and Colin L Mallows. A method for comparing two hierarchical clusterings. *Journal of the American Statistical Association*, 78(383):553–569, 1983. 3, 6
- [11] Ritwik Gupta, Richard Hosfelt, Sandra Sajeew, Nirav Patel, Bryce Goodman, Jigar Doshi, Eric Heim, Howie Choset, and

- Matthew Gaston. xbd: A dataset for assessing building damage from satellite imagery, 2019. 4, 6
- [12] Hanxiang Hao, Sriram Baireddy, Emily R. Bartusiak, Latisha Konz, Kevin LaTourette, Michael Gribbons, Moses Chan, Mary L. Comer, and Edward J. Delp. An attention-based system for damage assessment using satellite imagery, 2020. 2, 8
- [13] Richard D Hipp. SQLite, 2020. 6
- [14] Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of Classification*, 2(1):193–218, 1985. 3, 6
- [15] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020. 2, 4
- [16] Niloofar Khodaverdi, Heidar Rastiveis, and Arash Jouybari. Combination of post-earthquake lidar data and satellite imagery for buildings damage detection. 3:12–20, 2019. 8
- [17] Priya Kuckreja, Rohit Sharma, et al. Geochat: Grounded large vision-language model for remote sensing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 8
- [18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kütler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020. 2
- [19] Fan Liu, Delong Chen, Zhangqingyun Guan, Xiaocong Zhou, Jiale Zhu, Qiaolin Ye, Liyong Fu, and Jun Zhou. Remoteclip: A vision language foundation model for remote sensing. *IEEE Transactions on Geoscience and Remote Sensing*, 62, 2024. 6, 7
- [20] James MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, pages 281–297, Berkeley, CA, USA, 1967. University of California Press. 4, 6
- [21] OpenAI. Gpt-4: Large multimodal model. Available at <https://openai.com/research/gpt-4>, 2025. 2, 3, 6
- [22] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. 7
- [23] Keshav Santhanam, Omar Khattab, Christopher Potts, and Matei Zaharia. Plaid: An efficient engine for late interaction retrieval. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM)*, page 1747–1756, 2022. 2, 4
- [24] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734, Seattle, United States, 2022. Association for Computational Linguistics. 5
- [25] Jan Luca Scheerer, Matei Zaharia, Christopher Potts, Gustavo Alonso, and Omar Khattab. Warp: An efficient engine for multi-vector retrieval. In *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2025. 2, 4
- [26] Junghoon Seo, Seungwon Lee, Beomsu Kim, and Taegyun Jeon. Revisiting classical bagging with modern transfer learning for on-the-fly disaster damage detector, 2019. 8
- [27] Robert R. Sokal and Charles D. Michener. A statistical method for evaluating systematic relationships. *University of Kansas Science Bulletin*, 38:1409–1438, 1958. 4, 7
- [28] Nikolaos Tziokas, Nikolaos Soulakellis, and Konstantinos Topouzelis. Use of object based image analysis in very high-resolution images to evaluate buildings damage after an earthquake: The case of vryssa settlement, lesvos island. 2018. 8
- [29] Nguyen Xuan Vinh, Julien Epps, and James Bailey. Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance. *Journal of Machine Learning Research*, 11:2837–2854, 2010. 3, 6
- [30] Xiao Wang, Di Wang, Tiegang Sun, Jianhui Dong, Luting Xu, Weile Li, Shaoda Li, Peilian Ran, Jinxi Ao, Yulan Zou, Jing Wang, and Xinnian Zeng. Dual path attention network (dpanet) for intelligent identification of wenchuan landslides. *Remote Sensing*, 15(21), 2023. 1, 8
- [31] Joseph Z. Xu, Wenhan Lu, Zebo Li, Pranav Khaitan, and Valeriya Zaytseva. Building damage detection in satellite imagery using convolutional neural networks. *ArXiv*, abs/1910.06444, 2019. 2, 3, 6, 8
- [32] Parviz Zeaieanfirouzabadi, Saeed Saroei, and Esmat Madanian. Flood damage assessment using remote sensing technologies. *Applied Geography Conferences*, pages 291–299, 2012. 8
- [33] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. 6
- [34] Hongzhe Zhang, Xiaohang Zhao, and Xiao Fang. Proactive resource request for disaster response: A deep learning-based optimization model. *arXiv preprint arXiv:2307.16661*, 2023. 8

A. Appendix

A.1. DamageScope Prompt

Listing 4 shows the prompt used to elicit JSON-formatted output from GPT-4o during ingestion.

Listing 4. Extended GPT-4o prompt for structured damage extraction.

```
You are a damage assessment expert.
Given a satellite image of a property, analyze
visible changes and return a JSON with the
following structure:
```

```
{
  "roof_damage": {
    "prediction": <bool>, "reason": "<why>"
  },
  "structural_damage": {
    "prediction": <bool>, "reason": "<why>"
  },
  "lawn_damage": {
    "prediction": <bool>, "reason": "<why>"
  },
  "water_damage": {
    "prediction": <bool>, "reason": "<why>"
  },
  "fire_smoke_damage": {
    "prediction": <bool>, "reason": "<why>"
  },
  "occupiability": {
    "prediction": "<habitable|
      partially_habitable|uninhabitable>",
    "reason": "<why>"
  },
  "damage_level": {
    "prediction": <0-3>, "reason": "<summary>"
  }
}
```

Guidelines:

- Only describe what is visually observable.
- Avoid guessing causes.
- Provide concise reasoning sentences per field.
- Maintain consistent JSON keys and order.

A.2. Example Property-Level Output

Listing 5 extends the JSON schema in the main paper (Listing 1), showing a full record with multiple fields and reasoning strings.

Listing 5. Extended example of structured property-level damage output.

```
{
  "property_id": "A12-487",
  "disaster_event": "Wildfire",
  "roof_damage": {
    "prediction": true,
    "reason": "Significant portions of the roof shingles
      ↪ are missing."
  },
  "structural_damage": {
    "prediction": true,
    "reason": "Walls exhibit dark scorch lines and
      ↪ foundation cracking."
  },
  "lawn_damage": {
    "prediction": true,
    "reason": "Vegetation fully charred and debris
      ↪ scattered across yard."
  },
  "water_damage": {
    "prediction": false,
    "reason": "No visible signs of water pooling or
      ↪ discoloration."
  },
  "fire_smoke_damage": {
```

```
    "prediction": true,
    "reason": "Heavy smoke residue observed on roof and
      ↪ facade surfaces."
  },
  "occupiability": {
    "prediction": "uninhabitable",
    "reason": "Structural and fire damage render the house
      ↪ unsafe."
  },
  "damage_level": {
    "prediction": 3,
    "reason": "Major roof loss with likely internal damage.
      ↪ "
  }
}
```

A.3. SQL and Vector Storage Examples

Below we demonstrate how structured and unstructured fields from the JSON are stored separately in DamageScope’s dual-store architecture.

Listing 6. Simplified SQL row stored in the structured store.

```
{
  "property_id": "A12-487",
  "roof_damage": true,
  "structural_damage": true,
  "lawn_damage": true,
  "water_damage": false,
  "fire_smoke_damage": true,
  "occupiability": "uninhabitable",
  "damage_level": 3
}
```

Listing 7. Corresponding vector-store entry storing descriptive reasoning.

```
{
  "property_id": "A12-487",
  "roof_damage": "Significant portions of the roof shingles
    ↪ are missing.",
  "structural_damage": "Walls exhibit dark scorch lines and
    ↪ foundation cracking.",
  "lawn_damage": "Vegetation fully charred and debris
    ↪ scattered.",
  "fire_smoke_damage": "Heavy smoke residue observed on
    ↪ surfaces.",
  "occupiability_reason": "Structural and fire damage
    ↪ render the house unsafe.",
  "summary": "Major roof loss with likely internal damage."
}
```

This separation allows DamageScope to resolve both factual and explanatory queries efficiently, i.e. SQL queries for numeric/categorical attributes, vector queries for semantic reasoning.